Project Name: Chess Mini (Dabaa), 6*6

```cpp
#include <iostream>
#include <cstdlib>
#include <cctype>
using namespace std;

const int N = 6;
char board[N][N];

bool inside(int r, int c) {
    return r >= 0 && r < N && c >= 0 && c < N;
}

bool isWhitePiece(char p) { return p >= 'A' && p <= 'Z'; }
bool isBlackPiece(char p) { return p >= 'a' && p <= 'z'; }


bool ownedBy(char p, bool white) {
    if (p == '.') return false;
    return white ? isWhitePiece(p) : isBlackPiece(p);
}

void setupBoard() {
    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++)
            board[i][j] = '.';

    // Black (upore)
    board[0][0] = 'r'; board[0][3] = 'k'; board[0][5] = 'r';
    for (int j = 0; j < N; j++) board[1][j] = 'p';

    // White (niche)
    board[5][0] = 'R'; board[5][3] = 'K'; board[5][5] = 'R';
    for (int j = 0; j < N; j++) board[4][j] = 'P';
}

void printBoard() {
    cout << "\n     1  2  3  4  5  6   (Col)\n";
    cout << "   ----------------------\n";
    for (int i = 0; i < N; i++) {
        cout << " " << i + 1 << " | ";
        for (int j = 0; j < N; j++)
```

```
            cout << board[i][j] << "  ";
        cout << "|\n";
    }
    cout << "   ----------------------\n";
    cout << " (Row)\n\n";
}


bool pathClear(int fr, int fc, int tr, int tc) {
    int dr = (tr > fr) - (tr < fr);
    int dc = (tc > fc) - (tc < fc);
    int r = fr + dr, c = fc + dc;
    while (r != tr || c != tc) {
        if (board[r][c] != '.') return false;
        r += dr;
        c += dc;
    }
    return true;
}


bool validMove(int fr, int fc, int tr, int tc, bool white) {
    if (!inside(fr, fc) || !inside(tr, tc)) return false;
    if (fr == tr && fc == tc) return false;

    char p = board[fr][fc];
    if (!ownedBy(p, white)) return false;

    char target = board[tr][tc];
    if (ownedBy(target, white)) return false;
    int dr = tr - fr;
    int dc = tc - fc;
    char type = toupper(p);

    if (type == 'R') {
        return (dr == 0 || dc == 0) && pathClear(fr, fc, tr, tc);
    }
    if (type == 'K') {
        return abs(dr) <= 1 && abs(dc) <= 1;
    }
    if (type == 'P') {
        int dir = white ? -1 : 1;
        int startRow = white ? 4 : 1;
```

```
        if (dc == 0) {
            if (dr == dir && target == '.') return true;
            if (dr == 2 * dir && fr == startRow && target == '.' &&
                board[fr + dir][fc] == '.')
                return true;
            return false;
        }

        if (abs(dc) == 1 && dr == dir && target != '.') return true;
        return false;
    }
    return false;
}

bool findKing(bool white, int &kr, int &kc) {
    char k = white ? 'K' : 'k';
    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++)
            if (board[i][j] == k) {
                kr = i;
                kc = j;
                return true;
            }
    return false;
}


bool attacked(int r, int c, bool byWhite) {
    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++)
            if (ownedBy(board[i][j], byWhite) && validMove(i, j, r, c,
byWhite))
                return true;
    return false;
}


bool inCheck(bool white) {
    int kr, kc;
    if (!findKing(white, kr, kc)) return false;
    return attacked(kr, kc, !white);
}
```

```
bool legalMove(int fr, int fc, int tr, int tc, bool white) {
    if (!validMove(fr, fc, tr, tc, white)) return false;

    char moved = board[fr][fc];
    char saved = board[tr][tc];
    board[tr][tc] = moved;
    board[fr][fc] = '.';
    bool ok = !inCheck(white);
    board[fr][fc] = moved;
    board[tr][tc] = saved;
    return ok;
}

bool hasLegalMove(bool white) {
    for (int a = 0; a < N; a++)
        for (int b = 0; b < N; b++)
            if (ownedBy(board[a][b], white))
                for (int c = 0; c < N; c++)
                    for (int d = 0; d < N; d++)
                        if (legalMove(a, b, c, d, white)) return true;
    return false;
}

void makeMove(int fr, int fc, int tr, int tc, bool white) {
    board[tr][tc] = board[fr][fc];
    board[fr][fc] = '.';


    if (board[tr][tc] == 'P' && tr == 0) {
        board[tr][tc] = 'R';
        cout << "Pawn promoted to Rook!\n";
    }
    if (board[tr][tc] == 'p' && tr == N - 1) {
        board[tr][tc] = 'r';
        cout << "Pawn promoted to Rook!\n";
    }
}

int main() {
    setupBoard();
    bool whiteTurn = true;
```

```
    cout << "===== CHESS MINI (DABAA) 6x6 =====\n";
    cout << "P=Pawn  R=Rook  K=King\n";
    cout << "UPPERCASE = White, lowercase = Black\n";
    cout << "Move: fromRow fromCol toRow toCol (1-6)\n";
    cout << "Quit: 0 0 0 0\n";

    while (true) {
        printBoard();

        if (inCheck(whiteTurn)) cout << "*** CHECK! ***\n";

        if (!hasLegalMove(whiteTurn)) {
            if (inCheck(whiteTurn))
                cout << "CHECKMATE! " << (whiteTurn ? "Black" :
"White") << " wins!\n";
            else
                cout << "STALEMATE! Game draw.\n";
            break;
        }

        int fr, fc, tr, tc;
        cout << (whiteTurn ? "White" : "Black") << " turn > ";
        if (!(cin >> fr >> fc >> tr >> tc)) {
            cin.clear();
            cin.ignore(1000, '\n');
            cout << "Invalid input! Number dao.\n";
            continue;
        }

        if (fr == 0 && fc == 0 && tr == 0 && tc == 0) {
            cout << "Game quit.\n";
            break;
        }


        fr--; fc--; tr--; tc--;

        if (!legalMove(fr, fc, tr, tc, whiteTurn)) {
            cout << "Invalid move! Abar try koro.\n";
            continue;
        }

        makeMove(fr, fc, tr, tc, whiteTurn);
```

```
            whiteTurn = !whiteTurn;
        }

        return 0;
}
```

Output :

```
Problems  Output  Debug Console  Terminal  Ports

White turn > cd "c:\Users\shjoy\Downloads\" ; if ($?) { g++ chess_mini.cpp -o chess_mini } ; if ($?) { .\chess_mini }
Invalid input! Number dao.

    1  2  3  4  5  6   (Col)
  ----------------------
1 | r  .  .  k  .  r  |
2 | p  p  p  p  p  p  |
3 | .  .  .  .  .  .  |
4 | .  .  .  .  .  .  |
5 | P  P  P  P  P  P  |
6 | R  .  .  K  .  R  |
  ----------------------
(Row)
```

```
White turn > 5 1 4 1

    1  2  3  4  5  6   (Col)
  ----------------------
1 | r  .  .  k  .  r  |
2 | p  p  p  p  p  p  |
3 | .  .  .  .  .  .  |
4 | P  .  .  .  .  .  |
5 | .  P  P  P  P  P  |
6 | R  .  .  K  .  R  |
  ----------------------
(Row)
```

```
    1  2  3  4  5  6   (Col)
  ----------------------
1 | r  .  .  k  .  r  |
2 | .  p  p  p  p  p  |
3 | p  .  .  .  .  .  |
4 | P  .  .  .  .  .  |
5 | .  P  P  P  P  P  |
6 | R  .  .  K  .  R  |
  ----------------------
(Row)

White turn >
```